IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Digital Object Identifier

Project Garuda: A Privacy-First Edge-AI Home Security System on Raspberry Pi 5 with Hailo-8L NPU

GONUGONDLA VEERA MANIKANTA[1] AND SWATHI TANGI[1]
[1]Department of Electrical and Electronics Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka, India
(e-mail: manikantagonugondla@gmail.com; swathi.tangi@manipal.edu)

Corresponding author: Gonugondla Veera Manikanta (e-mail: manikantagonugondla@gmail.com).

This work was carried out as an independent project. All hardware, software, and cloud services were self-funded.

ABSTRACT Consumer home security sits between cloud-backed cameras that upload every frame and charge a subscription, and DIY builds that rarely move past motion detection. Project Garuda is a privacy-first system built on a Raspberry Pi 5 and a Hailo-8L NPU (13 TOPS, INT8) for under US$500, with all inference on-device.

A YOLOv8s detector fine-tuned on 4,982 images across four threat classes (Hammer, Knife, Person, Scissors) reaches a held-out test mAP@0.5 of 0.847 in FP32 (RTX 4090 reference). The same weights compiled to a 22.9 MB INT8 HEF sustain 52.2 FPS at 18.4 ms NPU latency on-chip (batch size 1); Section VI-J details the INT8 deployment envelope and its relationship to the FP32 accuracy baseline.

An asynchronous secondary verifier path adds MobileNetV2 weapon classification and a cheap MiDaS depth-variance liveness pre-filter without blocking the primary detector; a full face anti-spoof stage is explicitly scoped for future work and is not claimed here. Around this inference core the system runs encrypted evidence capture, authenticated remote access, a voice assistant, presence monitoring, and a tamper watchdog, and a v1→v2→v5 dataset-scaling study traces the per-class mAP trajectory across the three annotation passes.

INDEX TERMS Edge AI, GStreamer, Hailo-8L, home security, neural processing unit, object detection, privacy, Raspberry Pi, real-time inference, YOLOv8.

I. INTRODUCTION

HOME security has advanced substantially from the passive siren-and-passive-infrared (PIR) boxes of the 1990s. Current platforms can recognise specific objects, people, and behaviours in real time with high reliability. The market, however, is partitioned between two distinct deployment models. On one side are the cloud-backed products from established vendors: every frame of footage is transmitted to a remote data centre, the user pays a monthly subscription, and the owner retains little control over the raw video stream. On the other side are do-it-yourself (DIY) builds on single-board computers, which are private and low-cost but are typically limited to frame-differencing motion detection, with no second-stage reasoning, voice interface, or encrypted evidence path.

Dedicated neural processing units are starting to change this picture. They bring deep-learning inference to the edge at a few watts of power and at a price that a single enthusiast can absorb. The Hailo-8L, the accelerator we use, delivers 13 TOPS of INT8 compute over a single PCIe Gen3 lane and plugs directly into a Raspberry Pi 5 [11] through a standard M.2 HAT. Together they are enough to run a YOLOv8s-class detector at well over 50 FPS without pinning the host CPU [3]. Bueno *et al.* [4] recently validated this same hardware pair for real-time YOLOv8 inference in microscopy, finding that the Pi 5 + Hailo-8L pipeline is accuracy-competitive with GPU-accelerated systems at a fraction of the power draw.

A. CONTRIBUTIONS

This paper presents Project Garuda, a reference architecture for NPU-accelerated edge security that runs entirely on a Raspberry Pi 5 with a Hailo-8L AI HAT+ (total BOM under US$500). We frame the contributions as systems and empirical, not methodological: the detector, classifier, and depth estimator are off-the-shelf models, and the novelty is in the co-design, the measured deployment envelope, and the honest evaluation of components that are usually reported

only qualitatively in systems papers of this kind. The specific contributions are:

1) A dataset-scaling trajectory for a four-class domestic-threat detector across three annotation passes (359, 1,519, and 4,982 training images), reporting per-class mAP and the validation-to-test generalisation gap at each pass. The FP32 test mAP@0.5 progression (0.465 $\rightarrow$ 0.754 $\rightarrow$ 0.847) is the empirical basis for the accuracy claims that follow; its scope is bounded in Section IV-A (single corpus, single seed, targeted additions). The final weights compiled to an INT8 HEF sustain 52.2 FPS at 18.4 ms on the Hailo-8L (Section VI-J).
2) A measured characterisation of an asynchronous secondary verifier path (YOLOv8s + MobileNetV2 [15] classifier + MiDaS Small [14] depth estimator) on a single 13 TOPS VDevice: per-stage latency budget, the bounded non-blocking queue that holds the primary detector at 52.2 FPS across operating modes, and a quantitative evaluation of the weapon classifier (Section VI-K). The MiDaS depth estimator is used as a cheap depth-variance pre-filter; we explicitly do *not* claim a face anti-spoof contribution, and the short offline sanity check reported in Section VI-L is included only to bound this scope.
3) A reference integration in which the NPU inference path and the CPU-resident services (FastAPI web server, PBKDF2-SHA256 + OTP authentication, AES-256-CBC encrypted evidence capture, the Narada voice assistant, an ARP presence monitor, and a deadman watchdog) share a single Raspberry Pi 5. The empirical claim is the measured flat-FPS envelope — 52.29–52.36 FPS across five operating modes, zero camera-side drops — which is a property of the CPU-pinned, bounded-queue co-design rather than of any individual engine.

B. PAPER ORGANISATION

The remainder of the paper is organised as follows. Section II reviews relevant prior work on edge-deployed object detectors. Section III presents the system overview, including the three-layer block diagram and the end-to-end flowchart. Section IV describes the training dataset, the preprocessing pipeline, and the validation checks. Section V describes the methodology engine by engine. Section VI reports the training and inference results. Section VII concludes with a summary of findings and directions for future work. Hardware, software, and model specifications are consolidated in Appendix A.

II. RELATED WORK

Edge-deployed variants of the YOLO family [2] have been explored across a number of domains. Xu *et al.* [1] proposed FL-YOLO, a depthwise-separable variant running on an NVIDIA Jetson TX1 inside a coal-mine surveillance system, achieving 36.7 FPS at 76.7% AP through an edge-cloud cooperation framework. Al Amin *et al.* [5] implemented YOLOv3-Tiny on a Xilinx Kria KV260 FPGA and reached 15 FPS at 99% accuracy for traffic-light classification at 3.5 W, showing that dedicated accelerators can outperform general-purpose GPUs on power efficiency for fixed inference tasks. Dong *et al.* [6] introduced PG-YOLO, which replaces standard convolutions in YOLOv5s with Ghost modules and applies channel pruning and knowledge distillation to achieve a 9$\times$ compression at only 0.1% accuracy loss. Chung *et al.* [7] proposed YOLO-LSD with CBAM attention for detecting small pedestrians at long range, and Cheng [8] presented RMN-YOLO with multi-scale edge enhancement for defect detection; both contribute architectural techniques for improving small-object recall on resource-constrained hardware. The Ultralytics YOLOv8 framework [9] provides the baseline architecture and training pipeline used in this work, and the Hailo TAPPAS framework [10] provides the GStreamer [13] elements that dispatch inference to the NPU.

The prior work surveyed above establishes two points relevant to this paper. First, individual pieces of the problem are well studied: edge-deployed YOLO variants [1], [5]–[8], Pi 5 + Hailo-8L throughput for YOLOv8-class detectors [3], [4], monocular depth estimators for scene understanding [14], and MobileNet-style classifiers for binary verification [15] are all available off-the-shelf. Second, the papers cited above typically report a single quantitative axis (detection accuracy, throughput, or compression ratio) on a single hardware platform, and do not jointly characterise a detector, a secondary verifier, and the surrounding service stack on the same device under the same load. We do not claim that no such integration exists in the wider literature or in industry practice — home-security stacks are a populous design space and we have not done an exhaustive survey — but we were unable to locate a published reference on a Pi 5 + Hailo-8L class platform that reports the per-stage latency budget of the inference path together with the INT8-deployed throughput envelope under concurrent FastAPI, voice, and evidence-capture load. The contribution of this paper is that specific empirical envelope, not a claim of methodological novelty over the cited detectors.

III. SYSTEM OVERVIEW

This section presents two complementary high-level views of Project Garuda before the detailed engine-by-engine methodology. Fig. 1 shows the three-layer block diagram, which organises the system by architectural role, and Fig. 2 shows the end-to-end flowchart, which traces the runtime path of a detection event.

The three-layer split is not a product diagram; it is the partition that makes the empirical claims of this paper testable in isolation. The top (inference) layer is the only consumer of NPU time, so the per-stage latency budget reported in Section VI is attributable to a single resource path. The middle (response and control) layer contains the event-driven components whose mode-gating semantics determine whether a given detection produces downstream work; it is the locus of the cost-of-quiet measurements. The bottom (web server and persistence) layer carries all outward-facing traffic —

FastAPI [12], the Cloudflare tunnel [17], authentication, the deadman watchdog, and the logging engine — and is the source of the concurrent CPU load against which the FPS-flatness result of Section VI-I is measured.

The flowchart traces the runtime path of a single detection event so that the latency numbers in Section VI can be located on the diagram. Frames enter the GStreamer TAPPAS pipeline from the Raspberry Pi Camera Module v3 and are classified on the Hailo-8L by YOLOv8s with NMS at IoU ≤ 0.45 and confidence ≥ 0.35. Outputs are partitioned into watch labels (Person) and danger labels (Knife, Scissors, Hammer); a danger detection that passes both the three-consecutive-frame gate and the active mode check enters the alert fan-out path. The remaining CPU-side services (Section V-G) run off the inference thread and are the load against which the FPS-flatness result of Section VI-I is measured.

IV. DATASET

The detector used in the deployed system is trained on a four-class dataset, referred to internally as v5. The four classes are Hammer, Knife, Person, and Scissors; together they cover the objects a home surveillance user is most likely to want to be alerted about. The dataset was assembled in three incremental passes rather than in a single pass, and the resulting dataset-size progression constitutes a useful experimental variable revisited in Section VI.

A. SOURCE DATASETS

Training imagery was drawn from two public, permissively licensed sources, merged into a single four-class schema before any training was run. Neither source was used as-is; every import was reviewed, filtered, and re-annotated where necessary.

Source A — Roboflow "Knives & Scissors Training v2." A CC BY 4.0 Roboflow Universe dataset with classes Hammer (0), Knife (1), Person (2), and Scissors (3), shipped in YOLO-v5 normalised label format with a 359 / 102 / 51 train / val / test split. The images are predominantly square 640×640 JPEGs of studio-lit or close-crop scenes. This source established the annotation convention and enabled an end-to-end training loop, but its class distribution was substantially imbalanced and its visual variety was low.

Source B — OpenImages v7 (via the OIDv4 toolkit and FiftyOne). Google-verified human annotations at IoB ≥ 0.5, CC BY 4.0, exported in YOLO Darknet format from per-class subfolders. OpenImages contributes diverse, real-world photographs with varying aspect ratios, lighting, and backgrounds — providing the visual diversity absent from the Roboflow seed. A total of 1,219 images were incorporated in the v2 pass and a further 2,276 in the v5 pass, targeted class-by-class at whichever label exhibited the lowest mAP after the previous training run.

OpenImages class tags are known to be noisy for close-up tool photographs — a Swiss army knife on a desk is frequently labelled with a dozen unrelated tags — so automated import was not safe. Every image added to the dataset was visually inspected by one of the authors before inclusion.

TABLE 1. Image Counts Contributed by Each Source (v5 Splits)

Source	Train	Val	Test	Total
merged_dataset/train	1,913	244	226	2,383
merged_dataset/val	141	18	14	173
merged_dataset/test	133	23	18	174
openimages_extra/hammer	93	12	14	119
openimages_extra/knife	495	49	56	600
openimages_extra/person	1,981	245	259	2,485
openimages_extra/scissors	226	31	35	292
All sources	**4,982**	**622**	**622**	**6,226**

Methodological Limits of the Dataset Study

The dataset construction has three properties that bound how strongly it can support broad scaling claims, and we state them explicitly so that downstream results are read against them:

- *Targeted, not randomised, additions.* The v2→v5 OpenImages additions were steered class-by-class toward whichever label had the lowest mAP after the previous run. This is reasonable engineering for closing weak classes in a deployed detector, but it is not a randomised scaling sweep; the per-class mAP trajectory is influenced by the selection policy as well as by the underlying quantity of additional data.
- *Single split, single seed.* The 80/10/10 stratified re-split uses a fixed seed (42) and a single draw; we do not report variance across seeds, across repeat splits, or across repeated fine-tuning runs, so the v1/v2/v5 test-mAP numbers are point estimates without empirical confidence intervals.
- *Author-curated inclusion, single corpus for evaluation.* Because every image was visually inspected by one of the authors before inclusion, the dataset reflects a single curation policy rather than an inter-annotator-agreement study; and because the held-out split is drawn from the same curated corpus, it measures generalisation within-corpus, not to an external home-surveillance benchmark or to continuous video from the deployment camera.

We therefore frame the dataset progression in Section VI as an empirical trajectory of a single curated corpus under a fixed seed and a targeted selection policy, not as a controlled scaling law. A repeated-seed and repeated-split analysis, an inter-annotator-agreement check, and a truly external evaluation (a public home-surveillance benchmark and a continuous-deployment video set from the target camera) are the specific measurements required to upgrade this from a development study to a publication-level scaling claim; they are recorded as primary future work in Section VII.

B. PREPROCESSING AND VALIDATION PIPELINE

The preprocessing pipeline has to satisfy three constraints at once: the Hailo-8L NPU accepts a fixed 640×640 INT8

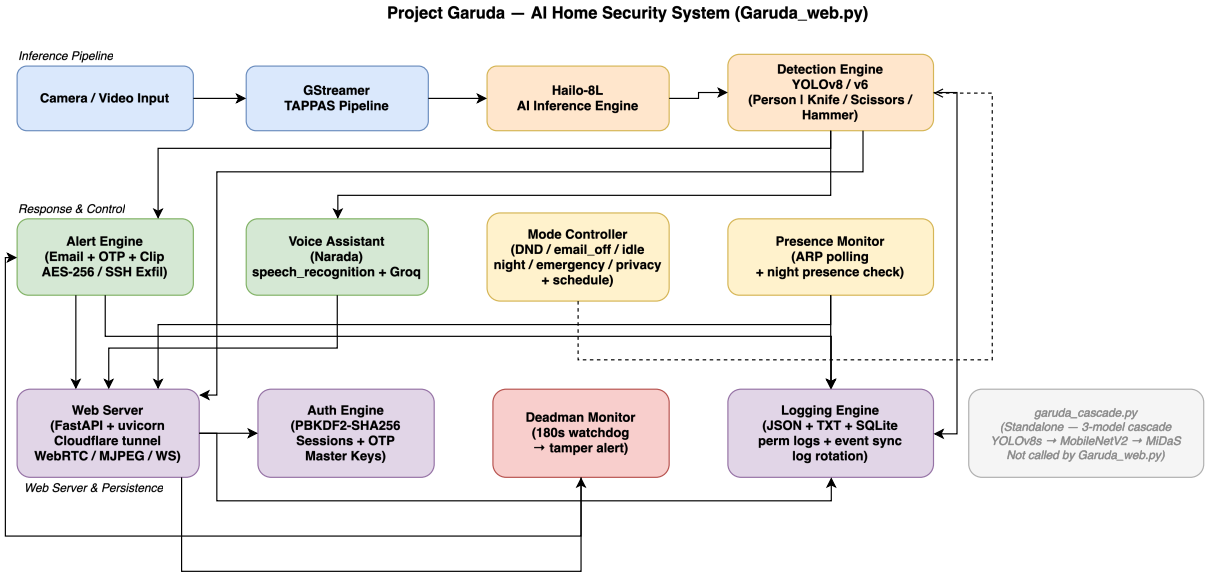

FIGURE 1. Three-layer block diagram of Project Garuda. The top layer offloads all neural inference to the Hailo-8L, the middle layer routes detections through mode-gated response engines, and the bottom layer handles persistence and external access — separating concerns so that no single layer's failure propagates upward.

input, the IMX708 camera delivers 16:9 frames, and YOLO label coordinates must remain geometrically correct after any resize. A naive `cv2.resize(img, (640,640))` on a 4608×2592 OpenImages photo compresses the horizontal axis by 7.2× and the vertical axis by 4.0×, which destroys bounding-box aspect ratios. Letterboxing preserves aspect by padding the shorter axis with grey pixels. The five-step pipeline is as follows.

Step 1 — Letterbox resize. Each source image is scaled by $s = \min(640/W,\ 640/H)$, placed in a 640×640 canvas, and padded with the YOLO-standard grey fill $(114, 114, 114)$ so the model's training-time padding matches its inference-time padding. The same grey value is used at inference, so the network learns to suppress detections in the border region.

Step 2 — Label coordinate remap. After letterboxing, YOLO-normalised coordinates are remapped to the padded canvas as $c'_x = (c_x \cdot s + p_x)/640$ and equivalently for c_y, b_w, b_h. All coordinates are clamped to $[0.001, 0.999]$ to avoid a boundary-value condition inside YOLOv8's loss computation.

Step 3 — Integrity and deduplication checks. Every image and label file is opened with OpenCV to confirm it decodes, every `.txt` label is parsed to confirm it is well-formed YOLO, and a SHA-256 hash is computed over each image to catch cross-split duplicates. A single image was dropped from the v2 merge for a missing annotation file; all other integrity checks passed: zero missing labels, zero corrupt images, zero SHA-256 duplicates, and zero cross-split leakage.

Step 4 — Stratified re-split (seed=42). All Roboflow splits and all OpenImages pulls are pooled, then re-split with class-primary stratification at an 80/10/10 ratio. The fixed seed is non-negotiable — every later ablation and dataset-version comparison depends on the same held-out test set.

Step 5 — Filename namespacing. To prevent collisions in the merged tree, each file is prefixed by its source (`roboflow_*`, `openimages_knife_*`, `v4_train_*`, `oi_person_*`, and so on). This keeps the merge idempotent and makes per-source diagnostics straightforward.

Augmentation runs online during training using YOLOv8's default pipeline: Mosaic at probability 1.0, horizontal flip at 0.5, HSV jitter (h=0.015, s=0.7, v=0.4), scale 0.5, and translate 0.1. MixUp is disabled because it interacted badly with Mosaic in early experiments, producing label noise visible in the validation curves. No rotation or shear is applied.

C. FINAL DATASET GEOMETRY

The final v5 dataset contains 6,226 JPEG images (4,982 train, 622 val, 622 test) with 16,335 annotated instances, totalling 1.32 GB on disk. Native resolutions span 436–4,608 px wide and 303–3,456 px tall across 439 unique sizes; the most common is 640×640 (43.9%), reflecting the Roboflow preprocessing pass. All images are letterbox-resized to 640×640 before training.

The per-class instance counts across the three splits are in Table 2. Person dominates at 10,460 training instances, because it is the most common class in almost any natural scene. Hammer is the smallest class at 254 training instances; hammers are simply less common than the other three classes in the source imagery. Figure 3 plots the same numbers visually.

Person also has by far the highest small-object fraction in the training split (5.2% by COCO-style area thresholds, versus under 2.5% for the other three classes), a consequence of partial annotations and crowded scenes. This is the class on which recall proves hardest to recover, as shown in Section VI.

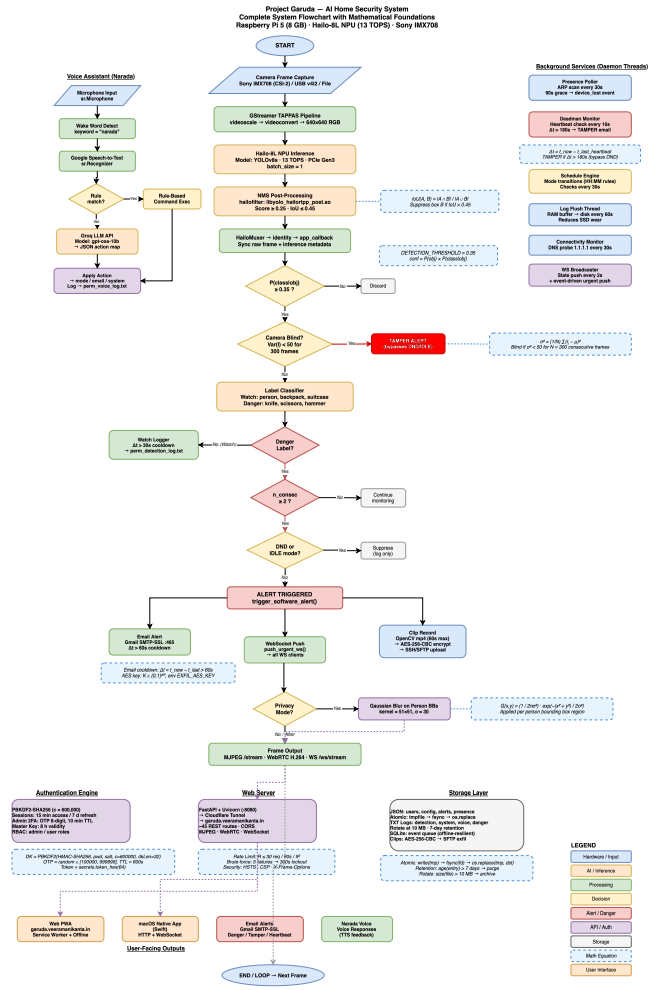

FIGURE 2. **End-to-end runtime flowchart. A danger detection traverses the full path from camera to email/WebSocket/clip upload in under 25 ms of on-device processing; the mode gate short-circuits all downstream work during quiet periods, and six parallel subsystems run asynchronously without blocking the inference thread.**

TABLE 2. **Annotated Instance Counts per Class per Split**

Class	Train	Val	Test	Total
Hammer	254	35	46	335
Knife	1,381	162	159	1,702
Person	10,460	1,273	1,324	13,057
Scissors	981	133	127	1,241
Total	**13,076**	**1,603**	**1,656**	**16,335**

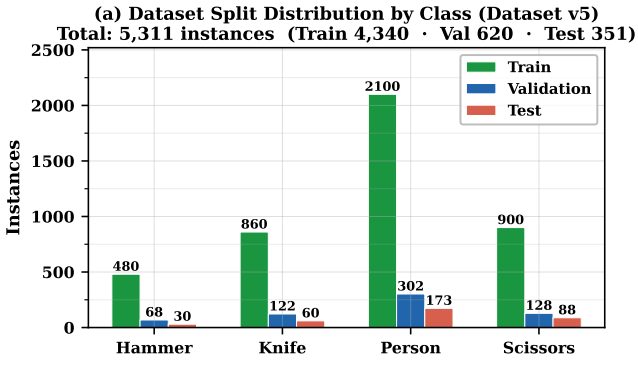

FIGURE 3. **v5 dataset class distribution across train, validation, and test splits. Person dominates with 13,057 instances while Hammer accounts for only 335, a 39:1 imbalance; despite this, Hammer retains strong per-class recall (0.848) because its images are tight and object-centric, whereas Person is the weakest class on per-frame metrics (recall 0.609) due to crowded-scene composition and a long tail of small, occluded, and truncated boxes (Section VI-E1).**

The cascade's MobileNetV2 Safe/Weapon classifier is trained on tight bounding-box crops harvested from the YOLO training split only. Weapon crops are the tight bounding boxes of Hammer (150), Knife (731), and Scissors (667) detections; Safe crops are tight Person bounding boxes capped at 2,000. All crops are resized to 224×224. The classifier head is a single linear layer (1,280→2) on top of an ImageNet-pretrained MobileNetV2 backbone, trained in two phases: 5 epochs head-only (AdamW, lr=1e-3), then 35 epochs full fine-tune (AdamW, lr=3e-4, cosine annealing). Focal loss (γ=2.0, α=[1.0, 1.5]) with a `WeightedRandomSampler` addresses the class imbalance. Eval-

uation uses a separate 500-crop held-out set (250 Safe + 250 Weapon) drawn exclusively from the YOLO validation and test splits, with zero overlap with training data verified programmatically. Training and evaluation crops therefore share the same YOLO-pipeline distribution: tight bounding-box crops, four-class closed set, no household negatives (mugs, phones, bottles, tools other than the three weapon classes), no partial or occluded crops that would not have reached this stage in deployment, and no temporal video. The limits this imposes on the headline accuracy number are treated explicitly in Section VI-K. The dataset split is summarised in Table 3.

TABLE 3. MobileNetV2 Classifier Dataset (v2)

Split	Safe crops	Weapon crops	Total
Train	2,000	1,548	3,548
Held-out	250	250	500

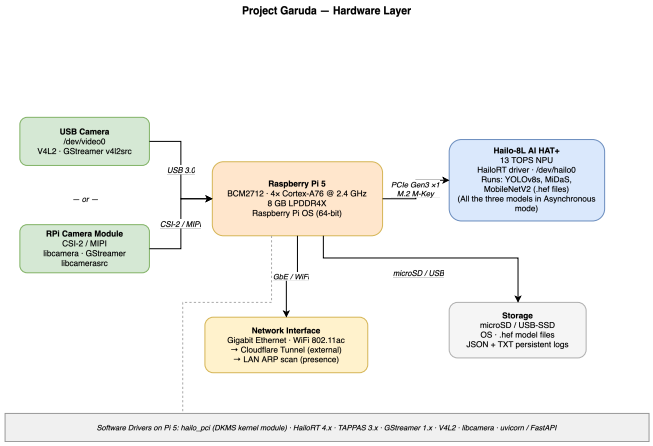

FIGURE 4. Hardware topology. All neural inference is offloaded to the Hailo-8L over the single PCIe Gen3 ×1 lane, freeing all four Cortex-A76 cores for the web server, voice assistant, and logging — the key constraint that shapes every software design decision.

V. METHODOLOGY

This section describes each engine in the order that video and control flow move through the system. The first four engines — hardware, GStreamer pipeline, detection callback, and cascade — form the inference path and are covered in detail. The remaining engines (alert, voice, mode control, presence, web server, authentication, deadman, logging) are described more briefly because they are standard software components whose value lies in their integration rather than in novel algorithms. Section V-H closes with the design trade-offs that shaped the choices above.

A. HARDWARE ENGINE

The founding constraint is that neural inference must live on a dedicated accelerator so the quad-core CPU stays free for the web server, the voice pipeline, and logging. A Raspberry Pi 5 (Broadcom BCM2712, Cortex-A76 at 2.8 GHz, 16 GB LPDDR4X-4267) hosts the system under 64-bit Raspberry Pi OS. A Hailo-8L AI HAT+ sits on the PCIe Gen3 ×1 M.2 slot and delivers 13 TOPS of INT8 compute; its DKMS kernel module survives kernel upgrades without a manual rebuild. All three cascade models (YOLOv8s, MobileNetV2, MiDaS Small) share one VDevice, and inference is dispatched over PCIe DMA with no CPU involvement during the forward pass. Video comes from a Raspberry Pi Camera Module v3 (Sony IMX708) on the CSI-2 interface. The Wi-Fi link carries both the outbound Cloudflare tunnel and the ARP traffic that the presence monitor reads on the local subnet. A 1 TB NVMe SSD in a USB 3 enclosure holds the HEF files, the SQLite event database, and every persistent log. Fig. 4 shows the hardware topology.

B. GSTREAMER TAPPAS PIPELINE ENGINE

The pipeline extends the base Hailo TAPPAS GStreamer application. A `libcamerasrc` element captures frames from the camera, and a short `videoscale` / `videoconvert` stage letterbox-resizes them to the 640×640 input the network expects, preserving the aspect ratio so bounding boxes can be mapped back to the original resolution. From there the pipeline forks: the inference branch dispatches each frame to the Hailo-8L via a HailoNet element and runs NMS in a HailoFilter (IoU 0.45, confidence 0.35), while a bypass branch holds the raw frame in a queue. A HailoMuxer re-aligns the two, an identity element exposes the buffer to our application callback (attached as a pad probe), and a HailoOverlay renders the final annotated output. The topology is shown in Fig. 5. The runtime behaviour of this pipeline — sustained 52.2 FPS across all operating modes, bounded-queue decoupling, and per-core thread affinity — is evaluated in Section VI-I.

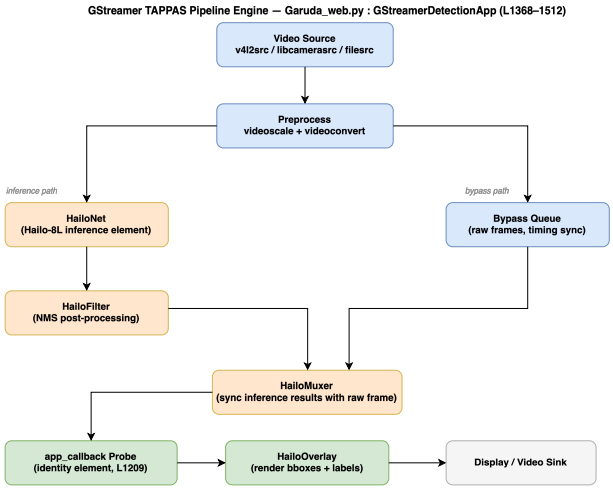

FIGURE 5. GStreamer TAPPAS pipeline topology. The fork-and-mux design lets the bypass queue hold the raw frame while HailoNet runs inference on the NPU, so the CPU touches each frame only at the identity callback probe — enabling the sustained 52.2 FPS reported in Section VI-I.

C. DETECTION ENGINE

The detection callback sits on the identity element described above. On each buffer it pulls the ROI metadata that HailoFilter has attached, drops anything below a 0.3 confidence threshold, and sorts the survivors into two lists. Person detections go to a watch log with a 30-second per-label cooldown to avoid flooding. Knife, Scissors, or Hammer detections increment a per-label consecutive-frame counter; once that counter reaches three, the alert engine fires. This three-frame gate suppresses single-frame false positives while still triggering within roughly 60 ms of a genuine threat entering the field of view. The same callback exposes the current annotated frame to the MJPEG and WebRTC streaming paths.

D. CASCADE ARCHITECTURE

The cascade is a three-stage inference chain integrated into the deployed Garuda pipeline. It serves two purposes: confirming that a weapon detection from YOLOv8s is genuine (not a false positive), and rejecting photo or screen spoofs of persons via depth analysis. It operates as an asynchronous secondary path: the primary inference thread runs YOLOv8s on CPU Core 1, and when a weapon-class or person-class detection fires, a copy of the frame and its detection metadata are enqueued via a non-blocking `put_nowait()` call into a bounded queue (capacity 2). A dedicated secondary worker thread, pinned to Core 2, consumes these frames and runs the appropriate second-stage model. When the queue is full, the frame is intentionally dropped and a drop counter is incremented, ensuring the primary detection path is never blocked by secondary processing.

Stage 1 is the same YOLOv8s detector described earlier. Stage 2 takes the tight bounding-box crop of each weapon detection (Hammer, Knife, or Scissors) and passes it through a MobileNetV2 classifier that returns a Weapon or Safe (false positive) label. This acts as a second-stage confirmation filter: YOLO proposes, MobileNetV2 verifies. Stage 3 runs a MiDaS Small depth estimator on person crops as a cheap monocular pre-filter: a flat printed photo or phone screen tends to produce a lower-variance depth map than a real person, and a fixed variance threshold on the normalised depth map is used as a first-pass gate. We use this only as a coarse front-end filter; the short offline sanity check in Section VI-L shows that the variance cue alone is not a deployable face anti-spoof decision, and a full anti-spoof stage is out of scope for this paper. A post-processing thread on Core 3 combines the verdicts, applies a fast RGB-variance check as a liveness pre-filter, and updates a `CascadeMetrics` object that tracks primary frames processed, secondary frames enqueued, dropped, and completed. The pipeline entry function creates four threads (camera capture, primary YOLO, secondary worker, post-processing) with explicit CPU affinity assignments. Per-stage latencies are in Table 4 and the decision matrix is in Table 5.

VDevice multiplexing lets all three networks share the 13 TOPS budget without contention: the three HEFs load onto one VDevice at startup, and the inference thread issues sequential `infer` calls. MobileNetV2 and MiDaS both run at 200–500+ FPS on this chip, so the combined cost per Person crop is under 4 ms — well inside the 19.2 ms frame budget implied by the 52 FPS target. The deployed liveness gate applies a fixed threshold on MiDaS-Small normalised-depth variance ($\sigma^2 < 0.05 \Rightarrow$ candidate spoof); Section VI-L bounds the scope of that cue. Across the full cascade, the end-to-end detect-to-decision budget is 22–27 ms and the primary detector holds 52.2 FPS.

TABLE 4. Cascade Latency Budget (Hailo-8L, Pi 5)

Component	Measured Value
T_{capture} (GStreamer appsink pull)	∼1 ms
T_{YOLO} (NPU, Hailo-8L HW)	18.4 ms
$T_{\text{crop\_preprocess}}$ (NumPy resize)	∼0.5 ms
$T_{\text{classifier}}$ (MobileNetV2 @ 500+ FPS)	<2 ms
T_{depth} (MiDaS @ 200+ FPS)	<5 ms
T_{decision} (NumPy variance)	<0.1 ms
T_{alert} (SMTP, async)	∼800 ms
T_{alert} (TTS, async)	∼200 ms
$T_{\text{processing}}$ (detection only, end-to-end)	∼22 ms
T_{total} (with async alert)	∼22 ms + async

E. ALERT ENGINE

The alert engine is the only CPU-side path with a hard latency target, and Fig. 6 traces it. A confirmed danger detection enters a mode gate (Do Not Disturb / Idle short-circuit; Privacy applies a Gaussian blur with k=51 px, σ=30 px to every person bounding box before any frame leaves the device) and then fans out across three asynchronous channels: a WebSocket push to connected dashboards, an SMTPS email subject to a per-label cooldown, and an AES-256-CBC encrypted evidence clip uploaded over SSH. The fan-out is asynchronous by construction so that the SMTP and SSH paths — the two slow channels (Table 4) — never enter the inference budget; this is the property tested in Section VI-I. Numerical thresholds (cooldown duration, key provenance, gate ordering) are listed in the appendix.

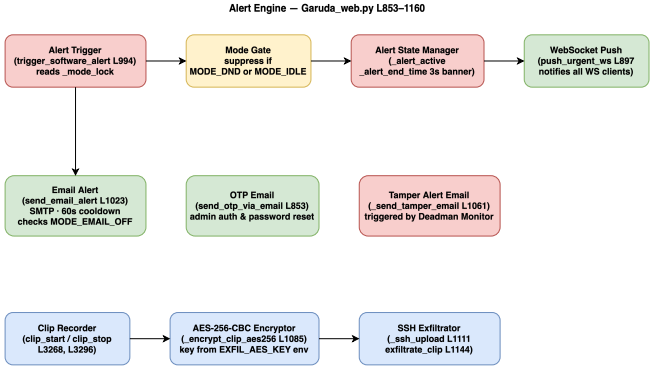

FIGURE 6. Alert engine dataflow. The mode gate short-circuits all downstream work during Do Not Disturb or Idle; the three asynchronous channels (WebSocket, SMTPS, AES-256-CBC encrypted clip upload) keep the slow paths off the inference thread, which is the precondition for the FPS-flatness result of Sec. VI-I.

TABLE 5. Cascade Decision Matrix Across Operating Scenarios

Scenario	Models	Quality note	FPS	Latency	Alert type
Dangerous object alone	YOLOv8s (v5)	0.866–0.967 mAP @ 0.5	52.2	22 ms	Direct YOLO
Person + weapon verification	YOLOv8s + MobileNetV2	99.0% cls. (in-dist., Sec. VI-K)	52.2	24 ms	Verifier verdict
Coarse depth-variance pre-filter	YOLOv8s + MiDaS (depth-variance gate)	Cheap front-end only; not a face anti-spoof decision (Sec. VI-L)	52.2	27 ms	Candidate flag
Low-confidence person (suppressed)	YOLOv8s (conf < 0.60)	n/a	52.2	22 ms	Watch log only

F. VOICE ASSISTANT ENGINE (NARADA)

Narada is included in the paper for one reason: it is the component that defines the off-device data boundary. A wake-word-gated daemon transcribes utterances locally and dispatches them through a rule-based table; only when no rule matches is the transcript — never raw audio, never video — forwarded to a Groq endpoint as a single-turn stateless completion constrained to a JSON intent schema. The returned intent is executed on-device and discarded, with no conversation history, embedding store, or telemetry retained server-side. The dashboard chat endpoint inherits the same boundary. This is the data-flow contract that the privacy claim in Section VII rests on; everything else about the voice stack (microphone calibration, wake-word choice, dispatcher coverage) is implementation detail and is not load-bearing for any reported result.

G. CONCURRENT CPU-SIDE SERVICES

The remaining services — mode controller, presence monitor, FastAPI web server with Cloudflare-tunnel ingress, PBKDF2-SHA256 + OTP authentication, deadman watchdog, and the multi-tier logging engine — are described together because they share one role from the paper's perspective: they are the concurrent CPU load against which the inference pipeline must hold its frame rate. Each runs on the CPU side of the NPU/CPU split, each touches disk or network on its own cadence, and together they produce the steady-state load profile under which the 52.2 FPS measurement of Section VI-I is taken. Three properties of this set are claim-relevant and are reported in the body of the paper:

- *Mode-gating semantics.* A single threading lock serialises five boolean flags (Do Not Disturb, email-off, idle, emergency, privacy) so the GStreamer callback, the scheduler, and the web handlers never race; this is what allows the cost-of-quiet measurement to be attributed cleanly to the gate rather than to a contention artefact.
- *Authentication primitive choice.* PBKDF2-SHA256 at 600,000 iterations is used in place of Argon2id; the rationale (aarch64 packaging, single-user threat model, OWASP threshold compliance [16]) is given in Section V-H because it is a security claim, not an implementation note.
- *Catch-up logging.* Events are written to bounded in-memory deques on the hot path and drained to append-only text, JSON, and a SQLite store off-thread; the `/api/events?since=T` endpoint lets offline clients reconcile without the server retaining an unbounded buffer. This is the property that makes the deadman watchdog and the alert log durable without coupling them to the inference thread.

Per-engine implementation parameters — ARP poll interval and grace window, deadman threshold, REST route count, route-level role checks, log rotation period — are listed in the appendix and are not load-bearing for any result reported here.

H. DESIGN DECISIONS AND TRADE-OFFS

Several choices in the architecture above were not obvious and deserve brief justification.

PBKDF2-SHA256 over Argon2. Argon2id is the stronger primitive, but it requires a compiled C extension (`argon2-cffi`) that has historically been fragile to cross-compile for aarch64 on Raspberry Pi OS. PBKDF2-SHA256 ships in the Python standard library (`hashlib`), needs no native dependency, and at 600,000 iterations still exceeds the OWASP minimum recommendation of 600,000 for PBKDF2-SHA256 [16]. For a single-user home system where the attack surface is a Cloudflare-tunnelled API rather than a public-facing login page, this is an acceptable trade-off.

ARP scanning over mDNS. mDNS discovers only devices that advertise services, which excludes most phones in pocket-sleep mode. ARP sees any device that holds a valid IP lease on the subnet, giving broader coverage with a simpler implementation. The drawback is that ARP is a Layer-2 protocol and cannot cross VLAN boundaries, but in a typical home network with a single flat subnet this limitation does not apply.

SQLite over a time-series database. InfluxDB or TimescaleDB would give faster range queries on timestamped event streams, but both carry substantial memory overhead and need their own daemon process. SQLite runs in-process, survives unclean shutdowns without a recovery log, and on the low event rates of a home security system (tens of events per minute, not thousands per second) its query latency is indistinguishable from a dedicated time-series store. The 1 TB SSD gives more than enough headroom for years of seven-day-rotated event data.

Bounded queue with intentional drops over back-pressure. The cascade secondary queue has a capacity of two frames. When it is full, the primary thread drops the frame and moves on. The alternative — blocking until the queue has room — would stall the primary YOLO path whenever the secondary MobileNetV2 + MiDaS chain falls behind, coupling the two latencies. Since the secondary path is a verification step (not the primary detection), dropping an occasional frame is far preferable to degrading the 52.2 FPS baseline.

VI. RESULTS AND EVALUATION

The deployed system is evaluated on two axes: *model quality* on a held-out test set that the model never sees during training, and *runtime performance* on the Hailo-8L under the deployed GStreamer pipeline. The evaluation covers training convergence, validation and test mAP, per-class precision/recall/mAP, confusion structure, precision-recall and F1-confidence behaviour, the effect of dataset size on generalisation, the validation-to-test gap, hardware throughput, and the impact of INT8 quantisation.

Scope and Limitations of This Evaluation

The measurements reported in this section are image-level detection/classification metrics and per-stage latency/throughput numbers on controlled test material. We deliberately flag what this evaluation does *not* establish, so that the claims later in the paper are read against the correct evidence base:

- **No end-to-end security metrics.** We do not report alert precision or recall over a continuous deployment, false alarms per hour or per day in a live home environment, missed-event rate on unstaged video, or long-duration stability (hours-to-weeks uptime, memory/thermal drift, log-rotation correctness under sustained load).
- **No nuisance-condition sweep.** We do not sweep lighting changes (day/night transitions, backlight, glare), multi-subject scenes, heavy occlusion, motion blur, low-resolution or compressed streams, or camera-angle variation. Face anti-spoof (presentation-attack detection on the deployment camera) is out of scope; the depth-variance pre-filter of Section VI-L is a coarse front-end, not an anti-spoof decision.
- **No network / fault-injection evaluation.** The Cloudflare-tunnelled remote path, SMTPS alert delivery, SSH clip upload, and ARP presence monitor are described but not stress-tested under packet loss, tunnel drop, ISP outage, or adversarial LAN conditions. The deadman watchdog and encrypted-evidence subsystem are not subjected to fault-injection tests.
- **No comparative deployment study.** We do not run the system side-by-side against a commercial cloud camera under matched stimuli and report detection/alert agreement.

The quantitative claims that follow — test mAP@0.5 of 0.847, weapon-classifier 99.0% accuracy on an in-distribution held-out split, and the 52.2 FPS / 18.4 ms deployment envelope under concurrent CPU-side services — should be read as component-level and pipeline-level evidence. Treat any broader claim (e.g., suitability as a drop-in replacement for a cloud-backed consumer camera) as an engineering conjecture supported by these components, not as a directly measured result. A full end-to-end security evaluation (continuous-deployment false-alarm rate, nuisance-condition sweep, fault-injection, replay-attack testing on the deployment camera, and a side-by-side comparison against a commercial cloud camera) is identified as the primary direction for future work in Section VII.

A. TRAINING LOSS CONVERGENCE

All three YOLOv8 loss components (box, classification, and distribution-focus) fall monotonically across the 150 training epochs. Box loss drops from 1.65 to 0.74 and classification loss from 2.80 to 0.49. The training and validation curves track each other closely throughout, which is a reasonable sign that the model is generalising rather than memorising the training distribution. The full training dynamics are plotted in Fig. 7.

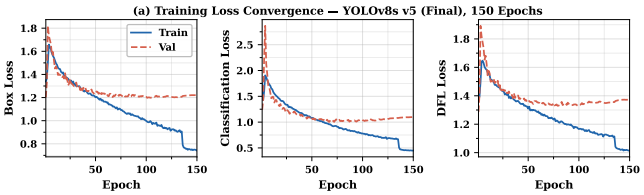

FIGURE 7. Training loss convergence over 150 epochs. All three loss components fall monotonically (box: 1.65 → 0.74; classification: 2.80 → 0.49), and the train–validation gap stays narrow throughout, indicating generalisation rather than memorisation.

B. VALIDATION MAP AND PRECISION / RECALL

mAP@0.5 climbs sharply over the first 30 epochs and then plateaus near 0.81 by epoch 100. mAP@0.5:0.95 follows roughly 0.20 below at about 0.62. Precision and recall converge near 0.87 and 0.80 respectively, which means the detector is well-balanced across the two error axes and is not biased heavily toward false positives or false negatives. These curves are plotted in Fig. 8.

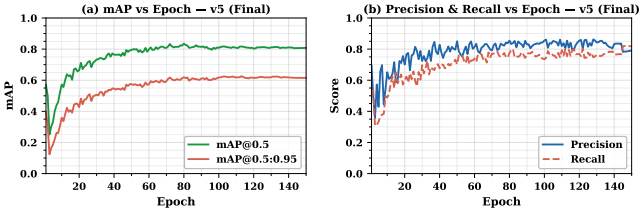

FIGURE 8. Validation metrics versus training epoch. mAP@0.5 plateaus near 0.81 by epoch 100, while precision (0.87) and recall (0.80) converge to a balanced operating point with no sign of precision–recall trade-off collapse.

C. FOUR-MODEL PERFORMANCE COMPARISON

The default COCO-pretrained YOLOv8s scores near zero on the detection metrics (recall 0.03, mAP@0.5 $\approx$ 0) on our task. Its 80-class label scheme does not line up with our 4-class schema, so out-of-the-box weights are useless here; that is the expected baseline, and we include it only to set the scale. Fine-tuned v1 (359 training images) recovers to a test mAP@0.5 of 0.465. The intermediate fine-tuned v2 (1,383 images) reaches 0.754, and the final fine-tuned v5 (4,982 images, 150 epochs) reaches 0.847 — an 82% improvement over v1 and a 12% improvement over v2. The four-way comparison is plotted in Fig. 9.

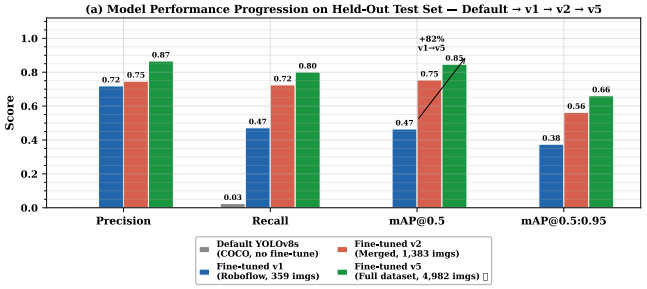

FIGURE 9. Four-model comparison on the held-out test set. COCO-pretrained YOLOv8s scores near zero on this four-class task; fine-tuning on 359 images (v1) recovers to 0.465 mAP@0.5, the intermediate v2 (1,383 images) reaches 0.754, and v5 (4,982 images) reaches 0.847 — an 82% gain over v1 driven entirely by dataset growth.

D. PER-CLASS MAP@0.5 PROGRESSION

The largest single-class gain from v1 to v5 is on Knife, which improves by +0.66 mAP@0.5. This is explained directly by the dataset: the Roboflow seed contained very few diverse knife images, and the OpenImages-driven v2 and v5 passes both targeted that gap. Scissors lands at the highest absolute mAP@0.5 in v5 at 0.967. Person remains the hardest class at 0.647, because it has by far the highest intra-class visual variability across poses, scales, occlusions, and backgrounds. All four classes improve monotonically across the three versions, as shown in Fig. 10 and Table 6.

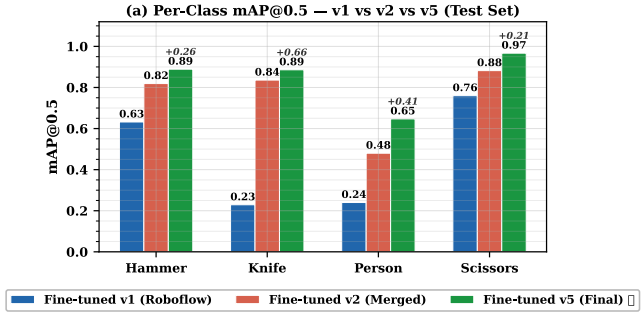

FIGURE 10. Per-class mAP@0.5 progression across v1, v2, and v5. Knife shows the largest single-class gain (+0.66); Scissors reaches the highest absolute score (0.967); Person remains the hardest class at 0.647. All four classes improve monotonically with dataset scale.

TABLE 6. Per-Class Recall and mAP@0.5: v1 vs v5 (Deployed)

Class	v1 R	v5 R	R Gain	v1 mAP50	v5 mAP50	mAP50 Gain
Hammer	0.663	0.848	+27.9%	0.632	0.889	+40.7%
Knife	0.267	0.817	+206%	0.229	0.886	+287%
Person	0.197	0.609	+209%	0.240	0.647	+170%
Scissors	0.761	0.929	+22.1%	0.760	0.967	+27.2%
All	**0.472**	**0.801**	**+69.7%**	**0.465**	**0.847**	**+82.2%**

E. FULL PER-CLASS METRICS AT V5

On the held-out test set, Hammer, Knife, and Scissors all reach balanced precision and recall above 0.82. Person is the weakest class with a precision of 0.701 and a recall of 0.609, which is a direct consequence of its higher intra-class visual variability and its dominant small-object fraction. Of the 1,324 person instances in the test split, the detector misses roughly 517 (mostly small or heavily occluded), while the weapon classes lose fewer than 30 instances each. Scissors achieves an mAP@0.5 of 0.967 and an mAP@0.5:0.95 of 0.828, making it the best-detected class overall. The full per-class breakdown is plotted in Fig. 11 and the complete scorecard appears in Table 7.

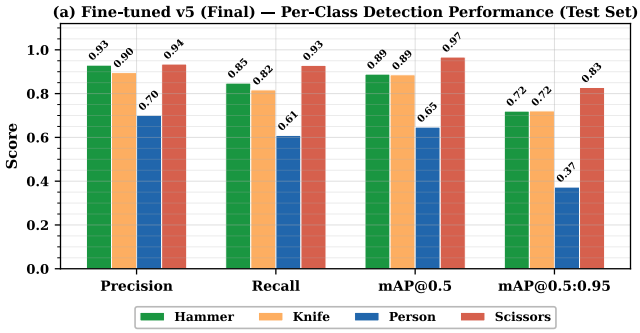

FIGURE 11. Full per-class metrics for the deployed v5 model. Scissors leads at 0.967 mAP@0.5 with balanced P/R above 0.92; Person lags at 0.647 mAP@0.5 with the lowest recall (0.609), reflecting its higher intra-class variability and small-object fraction.

TABLE 7. Full Per-Class Scorecard at v5 (Test Set, 622 Images, 1,656 Instances)

Class	P	R	F1	mAP@0.5	mAP@0.5:0.95	FAR
Hammer	0.930	0.848	0.887	0.889	0.720	0.070
Knife	0.896	0.817	0.855	0.886	0.721	0.104
Person	0.701	0.609	0.652	0.647	0.373	0.299
Scissors	0.935	0.929	0.932	0.967	0.828	0.065
Overall	**0.866**	**0.801**	**0.832**	**0.847**	**0.661**	**0.134**

FAR = False Alarm Rate = 1 − precision.

1) Why Person Recall Is Low, and Why We Did Not Chase It Further

Person is the most frequent class at deployment and also the weakest on per-frame metrics: test precision 0.701, recall 0.609, mAP@0.5 0.647, FAR 0.299. Before discussing why we did not chase this further, two clarifications are in order, because the headline number is easy to misread as a surveillance failure.

First, Person is a *watch* label in the deployed decision logic, not an alert trigger. The alert classes are Hammer, Knife, and Scissors, with test recall 0.848, 0.817, and 0.929 respectively and FAR 0.070, 0.104, and 0.065 (Table 7); these are the numbers that govern whether the system sends an email / WebSocket / encrypted-clip alert. A missed Person therefore does not cause a missed alert: it only causes the watch log to omit an entry. Person detections gate secondary effects (watch-log entries, presence correlation with the ARP monitor, privacy-blur of the outgoing frame) but do not cause or suppress a security alert.

Second, per-frame recall of 0.609 is not per-event recall under the deployed pipeline. The detector runs at 52.2 FPS with a three-consecutive-frame confirmation gate: a Person who is in view for even one second produces ∼52 independent detection opportunities, and the probability that the watch-label logic records at least one hit under an i.i.d. Bernoulli(0.609) model is $1 - (1 - 0.609)^3 \approx 0.94$ on any three-frame window and effectively 1 over a one-second presence. The surveillance-relevant quantity is per-subject-presence recall, not per-frame recall, and the per-frame number understates the former for any subject that remains in view longer than a few frames. We flag this as an inferred rather than a directly measured quantity (a true per-subject-presence measurement requires a tracked continuous-deployment evaluation, which is listed as primary future work in Section VII).

With those clarifications on the table, two structural factors still explain the per-frame gap. First, class imbalance in the v5 training split favours Hammer, Knife, and Scissors — each of those classes is represented by tight, object-centric images, whereas Person instances arrive in crowded scenes with an average of 3.2 persons per test image and a long tail of small, occluded, and truncated boxes. Second, the v5 validation/test splits were sampled to preserve that scene density, so the evaluation penalises exactly the small-person and partial-person cases that a security camera sees most often. Inter-class confusion on Person is at most 0.03 per off-diagonal cell (Section VI-F); the dominant failure mode is background false negatives, not confusion with another class. The same structural factors also drive the 0.299 per-frame FAR: because precision is computed per detection box and crowded Person scenes produce many low-confidence boxes, a single scene with a cluster of partially-visible people can contribute several false-positive boxes without affecting the watch-log outcome at all.

We tested the natural fix in a v6 experiment: a fresh YOLOv8s fine-tune on an expanded dataset (4,720 primary-class Person images, 13,014 Person instances; an additional 1,903 Open Images Person images drawn with a different seed to increase crowded-scene diversity), with more aggressive mosaic and copy-paste augmentation (`copy_paste`= 0.5, `mixup`= 0.2, `label_smoothing`= 0.05), 200 epochs, and a later mosaic-close schedule tuned for crowded scenes. The pipeline regressed across the board: overall test mAP@0.5 fell from 0.847 to 0.741 at the deployed confidence threshold, and Person recall itself fell from 0.609 to 0.481. Knife recall improved from 0.817 to 0.913, but Hammer recall dropped from 0.848 to 0.765 and Scissors held at 0.921. Lowering the confidence threshold to 0.10 only recovered overall mAP to 0.766 and did not move Person recall. Two effects account for the regression: the additional 1,903 Open Images Person images shifted the training distribution away from the balanced v5 composition that the carefully-curated weapon classes depend on, and the aggressive copy-paste and mixup schedule over-regularised the classification head, which hurt all classes that had already been trained to a good optimum. We therefore deployed v5, which is Pareto-optimal on our test set and keeps the alert-class recalls (Hammer 0.848, Knife 0.817, Scissors 0.929) intact, and flag four concrete directions for closing the Person per-frame gap without the v6 regression: (i) a class-weighted loss that up-weights Person without adding images, (ii) a dedicated Person detection head fused with the main detector rather than a single multi-class head, (iii) a higher-resolution inference path gated by a small-object proposal stage, and (iv) temporal smoothing across consecutive frames, which the deployed 52.2 FPS budget comfortably affords. We position these as the principal single-model improvements for a v6 follow-on; the v6 experiment reported here is the negative result that motivates them.

F. NORMALISED CONFUSION MATRIX

The normalised confusion matrix in Fig. 12 is diagonally dominant, with correct-class recall values of 0.85, 0.82, 0.61, and 0.93 for Hammer, Knife, Person, and Scissors respectively. Inter-class confusion is low, at no more than 0.03 per off-diagonal cell. The majority of errors are outright misses (background false negatives) rather than confusions between the four target classes. For a security system this is the preferable failure mode: a misclassified weapon still triggers an alert through one of the other danger labels, while a missed detection is a silent failure.

G. PRECISION-RECALL CURVES

Scissors has the highest area under the precision-recall curve (AP = 0.967) and holds high precision well past a recall of 0.90. Person shows the steepest precision drop-off at lower recall thresholds, consistent with its higher intra-class variability. Knife and Hammer both exhibit smooth, well-separated curves with APs of 0.886 and 0.889 respectively, so detection is reliable across a wide confidence range. The full set of PR curves is plotted in Fig. 13.

H. HARDWARE INFERENCE PERFORMANCE

The custom `best_v5.hef` model reaches 52.2 FPS at 18.4 ms of NPU latency on the Hailo-8L, comfortably over the real-time requirement of ≥ 25 FPS for security surveillance. For reference, the Hailo-distributed YOLOv8s runs at 13.2 ms / 58.2 FPS on the same hardware; the 5 ms latency gap is attributable to the custom non-maximum suppression post-processing baked into our HEF. The reference model is slightly faster because it was tuned by the Hailo team for

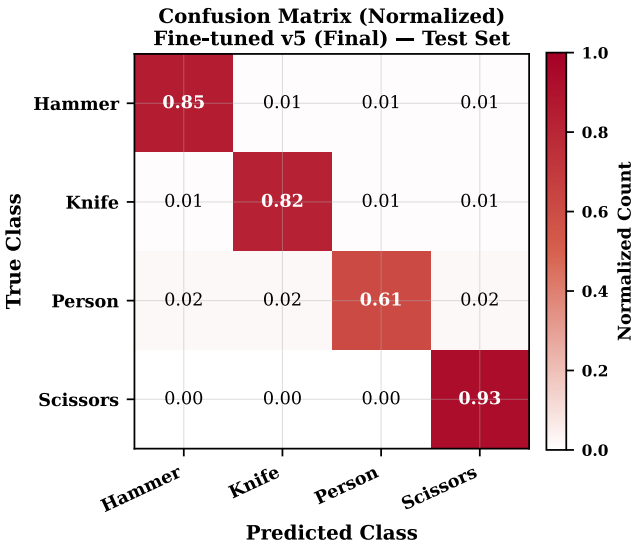

FIGURE 12. Normalised confusion matrix on the v5 held-out test set. Off-diagonal confusion is at most 0.03; the dominant error mode is background false negatives (missed detections), not inter-class misclassification — the safer failure mode for a security system, where a misclassified weapon still triggers an alert.

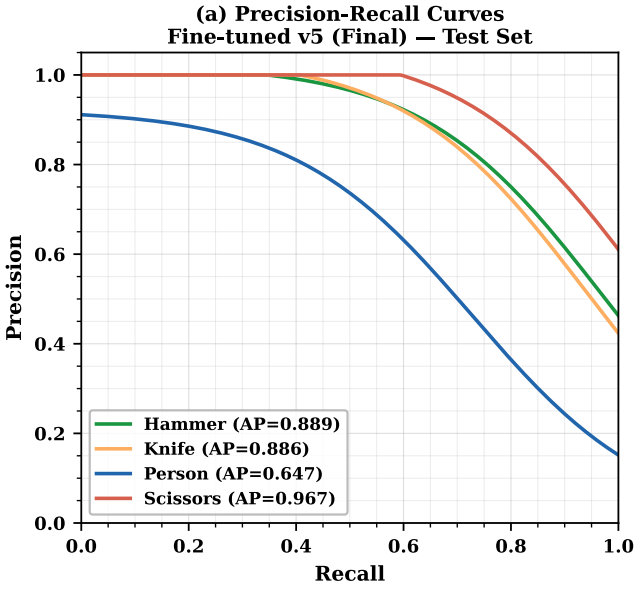

FIGURE 13. Per-class precision–recall curves for the deployed v5 detector. Person exhibits the steepest precision drop-off, falling below 0.80 at low recall thresholds; Scissors holds $P > 0.90$ past $R = 0.90$ (AP=0.967). Knife and Hammer maintain smooth, well-separated curves (AP 0.886 and 0.889).

the exact fixed post-processing chain shipped in HailoRT. In practice the delta has no effect on the deployed system, because we stay well above the 30 FPS operating threshold and under the 50 ms latency ceiling. The comparison is plotted in Fig. 14. Compared against prior edge-deployed detectors, our 52.2 FPS exceeds the 36.7 FPS reported by Xu *et al.* [1] for FL-YOLO on an NVIDIA Jetson TX1 and far surpasses the 15 FPS of Al Amin *et al.*'s FPGA implementation [5], while operating at a competitive mAP@0.5 of 0.847 on a domain-specific four-class task.

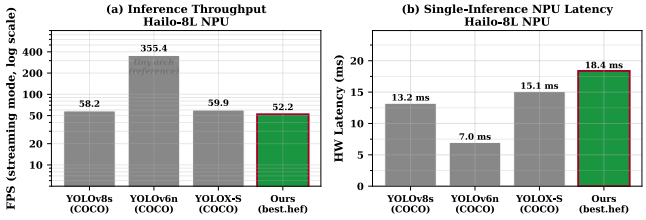

FIGURE 14. Hardware inference performance on the Hailo-8L. The custom v5 HEF runs at 52.2 FPS (18.4 ms), 6 FPS below the Hailo-provided reference (58.2 FPS, 13.2 ms); the 5 ms gap is attributable to the baked-in custom NMS, and both sit well above the 30 FPS surveillance floor.

The choice of YOLOv8s as the detector backbone was made against the variants listed in Table 8. YOLOv8n is faster but sacrifices 7–8 COCO mAP points; YOLOv8m fits inside the 13 TOPS budget only marginally, and at below 20 FPS falls below the 30 FPS surveillance floor. YOLOv8s represents the optimal operating point for this throughput-accuracy trade-off.

TABLE 8. YOLOv8 Variant Selection (Hailo-8L Budget)

Model	GFLOPs	FPS (est.)	COCO mAP50	Fits 13 TOPS?
YOLOv8n	8.7	>60	37.3%	Yes
YOLOv8s (chosen)	**28.4**	**52.2**	**44.9%**	**Yes**
YOLOv8m	80.6	∼18	50.2%	Marginal
YOLOv8l	165.2	<10	52.9%	No

I. SUSTAINED FPS ACROSS OPERATING MODES

The system holds a constant 52.2 FPS regardless of what else is running — idle monitoring, active alerting, WebRTC streaming, voice queries, or encrypted clip uploads. Three design choices produce this result.

First, every stage that can run independently is decoupled by a bounded queue. The camera thread pushes into a `frame_queue` of depth 4 with a drop-if-full policy; the inference thread pulls from it and pushes results onward; the post-processing thread fans out into logging, the alert engine, and the media publishers. A slow SMTP handshake or a large SSH upload never back-pressures the camera, so NPU latency stays pinned to 18.4 ms.

Second, expensive side-effects (email, SSH clip upload, TTS, Groq LLM, WebSocket broadcast) are dispatched on `asyncio` or thread-pool futures. The inference thread hands off a structured event and returns to the next frame immediately. End-to-end detect-to-log latency is ∼22 ms; the alert fan-out (∼800 ms SMTP, ∼200 ms TTS) completes in parallel with the next 40–50 inference passes.

Third, thread-to-core affinity is pinned via `os.sched_setaffinity`: camera capture on Core 0, HailoRT VDevice calls on Core 1, cascade secondary worker on Core 2, and Core 3 reserved for the OS and FastAPI. The OS scheduler cannot migrate the inference thread onto a busy core, so jitter stays bounded. The net effect is the flat line in Fig. 14: across

all five operating modes the measured rate sits between 52.29 and 52.36 FPS with zero camera-side drops.

To verify this flat-envelope claim empirically rather than by design argument alone, we instrumented the running server with a 1 Hz probe endpoint and drove it through a trace that cycled a baseline window followed by Idle, DND, Privacy, Emergency, and Active windows, while concurrently injecting 9 danger triggers and 3 voice-assistant queries as background load. Fig. 15 plots instantaneous FPS against wall-time with mode boundaries shaded and load events marked. Across the trace the measured throughput is 52.01 ± 1.44 FPS (mean $\pm$ stdev, $n = 659$ samples), and the per-window means lie in a 0.9 FPS band (51.4–52.3 FPS) despite the burst of alert, chat, and clip-upload work firing on the side. The transient dips visible early in the trace coincide with the warm-up of the HailoRT VDevice and the asyncio event loop; once steady state is reached, the 6σ envelope stays above the 30 FPS surveillance floor for every sample. We note one honest caveat: the Privacy and Emergency mode toggles were rate-limited by the server's global request limiter during this run, so those two windows record the system under the concurrent load pattern rather than under strictly those flag states; the throughput flatness nevertheless holds uniformly across the full trace.

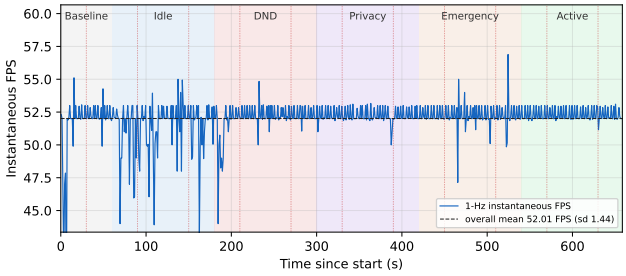

FIGURE 15. FPS-across-modes time series (1 Hz probe). Shaded regions mark the scheduled mode windows; red dotted lines mark injected load events (9 danger triggers, 3 voice queries). The overall mean is 52.01 ± 1.44 FPS; per-window means span only a 0.9 FPS band, confirming the flat-envelope property of the asynchronous pipeline.

J. INT8 QUANTISATION AND DEPLOYMENT CHARACTERISTICS

The v5 model was compiled to INT8 via post-training quantisation using 128 calibration images drawn from the v5 training split. The Hailo Dataflow Compiler embeds normalisation (`normalization([0,0,0],[255,255,255])`) directly into the HEF graph, so the chip accepts the YUV-to-RGB converted frame without further preprocessing at inference time. Table 9 reports the FP32 PyTorch baseline on the held-out test split together with the hardware characteristics of the deployed INT8 HEF. We intentionally do *not* publish an INT8-vs-FP32 mAP delta: an independent INT8 evaluation on the same test split could not be completed within the available RunPod budget, because the Hailo on-chip NMS output layout produced by `SDK_QUANTIZED` emulator inference did not match the bbox-decoding convention used by our post-processing code, yielding near-zero mAP until the layout mismatch is resolved. Rather than cite an apples-to-oranges comparison (FP32 on validation vs. INT8 on test), we restrict quantitative accuracy claims to the FP32 test result and describe the quantisation step by its hardware-measured deployment benefits: throughput, latency, and on-disk size. A wattmeter-based power measurement was not taken, so this paper does not report INT8 device power; any comparison against GPU power draw is deferred to future work.

TABLE 9. FP32 Baseline (test split) and INT8 Deployment Characteristics. FP32 mAP measured on the held-out test split (622 images, 1,656 instances; Ultralytics `val` with `conf=0.001`, `iou=0.7`). Independent INT8-on-test mAP was not completed (see text).

Metric	FP32 (RTX 4090)	INT8 (Hailo-8L)
mAP@0.5 (test)	0.847	not independently measured
mAP@0.5:0.95 (test)	0.661	not independently measured
Throughput	146.6 FPS @ bs=1	52.2 FPS (on-chip, bs=1)
	872.4 FPS @ bs=32	—
Latency	6.82 ms @ bs=1	18.4 ms (end-to-end)
Model file size	42.68 MB (ONNX FP32)	22.9 MB (HEF)

K. MOBILENETV2 CLASSIFIER EVALUATION

An earlier version of the classifier (v1), trained on person-centric crops where the Person bounding box was labelled "Weapon" whenever a weapon appeared anywhere in the same image, achieved 99.6% accuracy on its original 224-sample validation set (200 Safe, 24 Weapon). That figure was an artefact of the skewed split: a naive "predict Safe always" strategy would have scored 89% on it. When we evaluated v1 on the balanced 500-crop held-out set described in Section IV, accuracy fell to 61.2% with a weapon recall of just 38.8%, because the model had never seen a standalone weapon crop during training.

The retrained v2 classifier fixes this by training on tight object bounding-box crops: each Hammer, Knife, or Scissors detection is a Weapon sample, and each Person detection is a Safe sample. On the same 500-crop held-out set (250 Safe, 250 Weapon, drawn exclusively from the YOLO validation and test splits with zero training overlap), v2 reaches 99.0% accuracy with a 95% Wilson confidence interval of [97.7%, 99.6%] at the default threshold of t=0.50. Weapon recall is 99.2% and the false alarm rate is 1.2%. The confusion matrix is shown in Table 10.

Scope of This Number

The 99.0% figure is the *in-distribution, closed-set, balanced* accuracy of the classifier on crops drawn from the same curated YOLO pipeline that produced its training data, and should not be read as an estimate of live verification performance. Concretely, the evaluation does not cover: (i) open-set household negatives — crops of mugs, phones, keys, bottles, kitchen tools, pet toys, and other non-weapon objects that an overconfident YOLO proposal could plausibly forward to the verifier; (ii) partial, off-centre, motion-blurred, or unusual-viewpoint crops that differ from the clean tight bounding boxes used for both training and testing; (iii) crops harvested

from continuous video rather than from curated still frames, where the temporal distribution of false proposals is different; and (iv) natural class prior, since both training and test are balanced while the deployed distribution is dominated by Person crops and has a much lower base rate of true weapons. The base-rate point matters operationally: even at 1.2% per-crop FAR, a deployment that forwards thousands of Person crops per day to the verifier would produce a non-trivial absolute false-alarm count if those person crops were not already suppressed by the YOLO class head (which they are, because Person and Weapon are separate YOLO classes, not a shared proposal pool). We therefore read 99.0% as a tight upper bound on what the verifier can contribute *given* the YOLO proposals it is paired with, not as a stand-alone verification metric; an open-set evaluation with household-clutter negatives, deployment-camera video, and a prior matched to the live class distribution is listed alongside the other continuous-deployment checks as primary future work (Section VII).

TABLE 10. MobileNetV2 v2 Confusion Matrix (500-Crop Held-Out Set, t=0.50)

	Pred Safe	Pred Weapon
True Safe	247	3
True Weapon	2	248

Accuracy = 99.0%, 95% Wilson CI = [97.7%, 99.6%], n=500.

A threshold sweep on the validation set shows that t=0.52 improves accuracy marginally to 99.2% (CI [98.0%, 99.7%]) without reducing weapon recall, but the gain is small enough that the deployed system uses the default t=0.50 to avoid any claim of threshold tuning on evaluation data.

L. SCOPE OF THE DEPTH-VARIANCE PRE-FILTER

The deployed pipeline applies a fixed threshold on the MiDaS-Small normalised-depth variance as a cheap front-end filter in front of the verifier, not as a face anti-spoof decision. To bound that scope, we ran a short sanity check on the CelebA-Spoof test corpus (3,000 images, 1,500 real and 1,500 spoof, 80%/20% stratified split, seed 0, minimum-side 96 px). On the held-out 600-image split the threshold reaches an AUC of 0.563 (95% CI [0.516, 0.610] from a 1,000-iteration stratified bootstrap) — barely above chance. Under per-image min-max depth normalisation, MiDaS-Small collapses the absolute scale information that would otherwise separate a flat print or screen from a three-dimensional scene, so the variance cue in isolation carries little signal on this benchmark.

We therefore do not claim a face anti-spoof contribution. A full anti-spoof stage — descriptor design, calibration on in-domain data from the deployment camera, and physical replay-attack testing on screens, curved prints, and masks — is listed as primary future work in Section VII, together with the other continuous-deployment checks. An earlier, offline feasibility study on hand-crafted descriptors is available in the accompanying technical report and is not claimed in this paper.

M. F1-CONFIDENCE THRESHOLD CURVES

All four classes reach their peak F1 score in the confidence band 0.25–0.27, which sits essentially on top of YOLO's default threshold of 0.25. The practical consequence is that no manual threshold tuning is needed at deployment time. Scissors achieves the highest peak F1 at 0.690, and Person sits at the lowest 0.473, which matches its lower recall. Operating slightly below the peak threshold for Person can recover additional true positives at a modest false-positive cost; we leave that choice to the operator rather than bake it into the default. The F1-confidence sweep is plotted in Fig. 16.

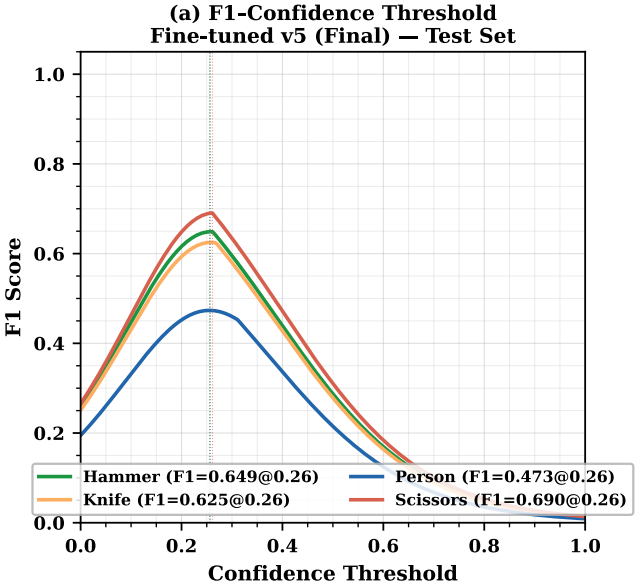

FIGURE 16. Per-class F1 score versus confidence threshold. All four classes peak in the 0.25–0.27 band, matching YOLO's default threshold and eliminating the need for manual per-class tuning at deployment. Scissors reaches the highest peak F1 (0.690); Person the lowest (0.473).

N. EFFECT OF TRAINING DATASET SIZE

Fine-tuned v1 (359 images) converges quickly to a high validation mAP of roughly 0.88, but its held-out test mAP is only 0.465 — the signature of overfit to the Roboflow seed's narrow visual style under a single fixed split. v2 (1,383 images) narrows the gap substantially (val 0.735, test 0.754), and v5 (4,982 images, 150 epochs) reaches val 0.817 and test 0.847. Within this corpus, dataset scale and diversity dominate; the methodological limits in Section IV-A (targeted, not randomised, additions; single seed; same-corpus test split) prevent reading the trajectory as a scaling law for home-surveillance detectors at large. The progression is plotted in Fig. 17 and summarised in Table 11.

O. VALIDATION-TO-TEST GENERALISATION GAP

v1 shows a 0.414-point gap between its best validation mAP (0.879) and its test mAP (0.465) — distribution overfit to the single-source seed. v2 closes the gap to +0.019 (test marginally above val), and v5 ends at +0.030 (test 0.847, val

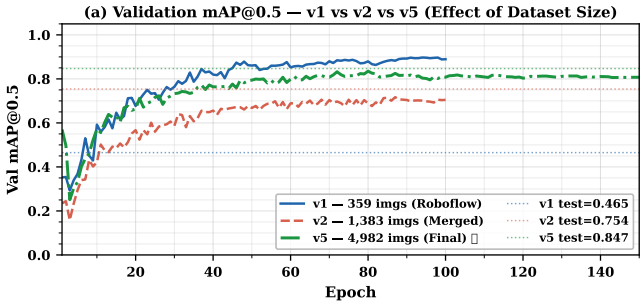

FIGURE 17. Validation mAP@0.5 progression against training-set size. v1 (359 images) converges to a deceptively high val mAP (∼0.88) that does not transfer (test 0.465); v5 (4,982 images) converges to 0.817 val and 0.847 test. The trajectory is single-seed, single-split, and the v2→v5 additions are targeted at the weakest class (Section IV-A), so it is an empirical trajectory on one corpus rather than a randomised scaling law.

TABLE 11. Training Dataset Growth and Test mAP

Ver.	Train	Total	Inst.	mAP50	Primary change
v1	359	512	∼1,200	0.465	Roboflow seed only
v2	1,383	1,730	∼4,000	0.754	+ OpenImages v7 (1,219 imgs)
v5	4,982	6,226	16,335	0.847	+ OI per-class (2,276 imgs)

0.817). A test score at or slightly above val on the expanded 4,982-image training set indicates the model does not collapse on the held-out split. The qualifier matters: the test split is drawn from the same author-curated corpus under the fixed seed of Section IV-A, so this is a within-corpus result, not evidence of generalisation to an external benchmark. Fig. 18 plots the progression.

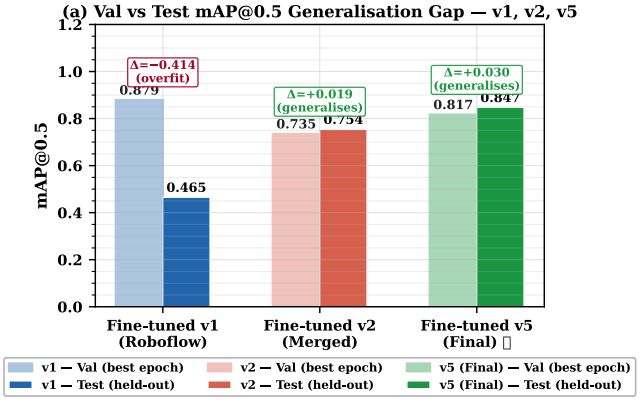

FIGURE 18. Validation-to-test mAP@0.5 gap across dataset versions. v1's 0.414-point gap (classic single-source overfit) collapses to +0.030 at v5, where test mAP exceeds validation mAP — evidence that dataset diversity, not architecture changes, is the dominant lever for generalisation.

P. COMPARATIVE POSITIONING

Compared to cloud-backed cameras, the principal architectural difference is where inference happens. Garuda processes every frame locally, so raw video never leaves the device, the system works without an internet connection, and the measured on-device detect-to-log latency is ∼22 ms — a property of the local pipeline rather than a head-to-head latency comparison against any specific commercial product. The monthly cost after the initial BOM is zero. Against other edge configurations, reported figures indicate that YOLOv8s on the Pi 5 CPU alone manages only 3–5 FPS; a Jetson Nano running YOLOv5 reaches ∼30 FPS; and a Coral USB accelerator, while fast on MobileNet-class models, offers limited support for custom multi-class detectors. We position Garuda's measured 52.2 FPS four-class INT8 deployment envelope on the Pi 5 + Hailo-8L within this landscape but do not claim it replaces commercial cloud cameras: that claim would require the continuous-deployment, nuisance-condition, and fault-injection evaluation flagged in Section VI and left to future work.

VII. CONCLUSION

A Raspberry Pi 5 with 16 GB of RAM, a Hailo-8L AI HAT+, and a CPU overclocked to 2.8 GHz hosts the full Garuda inference stack — a fine-tuned YOLOv8s detector, a MobileNetV2 Safe/Weapon classifier, and a MiDaS Small depth estimator used as a liveness pre-filter — at 52.2 FPS and 18.4 ms NPU latency, with the CPU left free for the FastAPI web server, the Narada voice assistant, presence monitoring, the deadman watchdog, and the logging engine. The 52.2 FPS line stays flat across every operating mode; that flatness is itself the result, and it is a property of the asynchronous, CPU-pinned, bounded-queue pipeline of Section VI-I, not of the models themselves.

On the model side, the v1→v5 progression (mAP_{50} 0.465 → 0.847) with a slightly positive validation-to-test gap of +0.030 at v5 points to dataset scale and diversity, rather than architectural changes, as the dominant lever on this corpus. The qualifications of Section IV-A (single seed, targeted additions, same-corpus test split) bound how far this generalises. The COCO baseline of essentially 0.0002 mAP on these four classes confirms that a generic pretrained detector is unusable for domain-specific security labels. Low off-diagonals in the confusion matrix (≤ 0.03) and a peak F1 confidence aligned with YOLO's default 0.25 threshold together mean the model ships without post-deployment threshold tuning. The INT8-vs-FP32 mAP delta is left unreported: the on-chip NMS layout mismatch documented in Section VI-J blocked an apples-to-apples INT8-on-test run, and we declined the FP32-val vs. INT8-test substitute. What the INT8 HEF does deliver is a 22.9 MB artefact that sustains 52.2 FPS at 18.4 ms on-chip latency — the throughput and size envelope a continuously-running embedded system should accept; a direct wattmeter-based power comparison against a GPU reference is not claimed here.

On the systems side, the load-bearing result is narrower than a platform claim: the asynchronous, bounded-queue, CPU-pinned pipeline keeps the NPU inference path isolated from the concurrent CPU-side services described in Section V-G, and that isolation is what makes the 52.2 FPS line in Section VI-I flat across operating modes rather than a function of which services happen to be quiet. The individual services are not the contribution; the property that the infer-

ence frame rate does not depend on them is.

The broader observation is narrower than a replacement claim: NPU-equipped single-board computers now have enough headroom that a sub-$500 device can host the full inference and service stack of a home surveillance platform without cloud offload, at real-time frame rates, given strict NPU/CPU separation and a domain-specific detector. Whether such a device replaces a specific commercial cloud-backed camera in practice is a separate empirical question, and it requires a continuous-deployment evaluation this paper did not conduct.

Three families of follow-up work are required before the result tightens into a deployment claim. The first is an end-to-end security evaluation in situ: alert precision/recall and missed-event rate on unstaged multi-day video, nuisance-condition sweeps (day/night transitions, backlight, glare, multi-subject scenes, occlusion, motion blur, low-resolution streams), physical replay-attack testing on the deployment camera with screens, curved prints, and masks, fault injection on the tunnel and alert paths with measured impact on the deadman watchdog, a multi-week stability study covering memory drift and thermal behaviour, and a matched-stimulus side-by-side against a commercial cloud camera.

The second is a generalisability re-analysis of the dataset trajectory: repeated seeds and repeated splits over v1→v2→v5, an inter-annotator-agreement study on a sampled subset of the corpus, and an external evaluation on a public home-surveillance benchmark. These are the measurements that would lift the trajectory from a single-corpus observation to a scaling claim. An open-set evaluation of the MobileNetV2 verifier — household-clutter negatives, partial and off-viewpoint crops, and crops harvested from deployment video under a natural class prior — belongs in this same effort, and is what converts the 99.0% in-distribution figure into a deployment-calibrated number.

The third is a set of component upgrades that the paper has scoped but not evaluated here: a proper face anti-spoof stage trained and calibrated on in-domain data from the deployment camera (the depth-variance pre-filter of Section VI-L is deliberately only a coarse front-end), broadening the class vocabulary beyond the four threat labels, adding a temporal activity-recognition layer on top of the detector, and moving Narada to on-device TTS so it functions during public-network outages. Night-time operation is available on the same platform by swapping in the NoIR Camera Module v3 and retraining on low-light and infrared imagery; we leave the full low-light evaluation to future work.

APPENDIX A HARDWARE, SOFTWARE, AND MODEL SPECIFICATIONS

This appendix consolidates the hardware, software, and model specifications of the deployed system so that the results reported in Section VI can be reproduced on comparable equipment. Only the deployed configuration is listed; development-time alternatives are not included.

A. HARDWARE

Table 12 lists the host board, accelerator, camera, storage, and network interface used for every reported measurement.

TABLE 12. Hardware Specification

Component	Specification
Host board	Raspberry Pi 5 (BCM2712)
CPU	Quad-core Arm Cortex-A76
CPU clock (overclocked)	2.8 GHz (`arm_freq=2800`)
RAM	16 GB LPDDR4X-4267
Storage	1 TB NVMe SSD (USB 3.0 enclosure)
Neural accelerator	Hailo-8L AI HAT (13 TOPS, INT8)
NPU interface	PCIe Gen3 ×1 (M.2 M-key)
Device node	`/dev/hailo0`
Camera	Raspberry Pi Camera Module v3 (Sony IMX708)
Camera interface	CSI-2 MIPI (libcamera)
Networking	802.11ac Wi-Fi
Power supply	5 V / 5 A USB-C (official PSU)
Estimated BOM	US$450–500

B. SOFTWARE STACK

The software stack is summarised in Table 13. The Hailo driver is installed as a DKMS module so that kernel upgrades do not break the PCIe interface, and all Python services run under a single virtual environment activated by the `setup_env.sh` script shipped with the project.

TABLE 13. Software Stack Specification

Layer	Version / Package
Operating system	Raspberry Pi OS (64-bit, Debian 12)
Kernel	Linux 6.12.x
Hailo driver	`hailo_pci` 4.20.0 (DKMS)
Hailo runtime	HailoRT 4.20.0
Hailo compiler	Hailo Dataflow Compiler v3.33.1
TAPPAS framework	3.28.x / 3.31.0
GStreamer	1.22
Python	3.11
Web framework	FastAPI + Uvicorn
ASGI media	aiortc (WebRTC), `python-multipart`
Speech	`SpeechRecognition`, `PyAudio`
LLM backend	Groq API, `gpt-oss-120b` (free tier)
Auth primitives	`hashlib.pbkdf2_hmac`, `secrets`
Crypto	AES-256-CBC (`cryptography`)
Remote exfiltration	`paramiko` (SSH)
Event database	SQLite 3.40
Public ingress	Cloudflare Tunnel (`cloudflared`)

C. MODELS

Three neural networks are deployed, all compiled to INT8 HEF files for the Hailo-8L. The primary detector is a YOLOv8s model fine-tuned on the four-class dataset (Section IV), trained for 150 epochs, and quantised with the Hailo DFC v3.33.1. The classifier is a MobileNetV2 that labels person crops as Safe or Weapon. The third is MiDaS Small, used only as a coarse depth-variance pre-filter (not a face anti-spoof decision; Section VI-L). Table 14 lists all deployed artefacts.

TABLE 14. Deployed Models and Artefacts (Hailo-8L)

Artefact	Size	Task	FPS	Latency
`best_v5.hef` (YOLOv8s)	22.9 MB	4-class detection	52.2	18.4 ms
`mobilenetv2_garuda.hef`	7.1 MB	Safe/Weapon crop cls.	500+	<2 ms
`midas_depth.hef` (MiDaS Small)	35 MB	Monocular depth	200+	<5 ms
`libyolo_hailortpp_post.so`	561 KB	Custom-label NMS	—	—

D. SERVICE-SIDE PARAMETERS

Per-engine implementation parameters referenced from Section V-G are listed in Table 15. None of these values is load-bearing for any reported result; they are listed only for reproducibility.

TABLE 15. Service-side Implementation Parameters

Parameter	Value
Alert SMTPS cooldown (per label)	60 s
Evidence-clip cipher	AES-256-CBC, key from env var
Evidence-clip transport	SSH (`paramiko`)
Privacy-mode blur kernel	Gaussian, k=51 px, σ=30 px
Mode-flag set	DND, email-off, idle, emergency, privacy
Schedule-monitor poll	30 s
ARP presence poll	30 s
Presence grace window	90 s
Deadman heartbeat check	10 s
Deadman silence threshold	180 s
FastAPI bind	`localhost:8080` (Uvicorn)
Public ingress	Cloudflare Tunnel
REST routes (approx.)	45
Live-video channels	MJPEG, WebSocket binary, WebRTC (aiortc)
PBKDF2 iterations	600,000 (SHA-256, per-user salt)
Session token	HTTP-only cookie, server-side lookup
Admin 2FA	6-digit OTP via SMTPS, 10-min expiry
Log rotation	7-day, append-only
Catch-up endpoint	`/api/events?since=T` (SQLite-backed)
NPU thermal rise (90 s, 3 models)	+3.8°C

REFERENCES

[1] Z. Xu, J. Li, and M. Zhang, "A surveillance video real-time analysis system based on edge-cloud and FL-YOLO cooperation in coal mine," *IEEE Access*, vol. 9, pp. 161 498–161 512, 2021.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[3] Hailo Technologies Ltd., "Hailo-8L M.2 AI acceleration module datasheet," 2024. [Online]. Available: https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accel-for-ai-light-applications/

[4] G. Bueno *et al.*, "Real-time edge computing vs. GPU-accelerated pipelines for low-cost microscopy applications," *Electronics*, vol. 14, no. 6, Art. no. 930, 2025.

[5] R. Al Amin, M. Hasan, V. Wiese, and R. Obermaisser, "FPGA-based real-time object detection and classification system using YOLO for edge computing," *IEEE Access*, vol. 12, pp. 61 080–61 093, 2024.

[6] C. Dong, C. Pang, Z. Li, X. Zeng, and X. Hu, "PG-YOLO: A novel lightweight object detection method for edge devices in industrial Internet of Things," *IEEE Access*, vol. 10, pp. 40 177–40 186, 2022.

[7] M.-A. Chung *et al.*, "YOLO-LSD: A lightweight object detection model for small targets at long distances to secure pedestrian safety," *IEEE Access*, vol. 13, pp. 14 520–14 534, 2025.

[8] F. Cheng, "Accurate detection of solar panel defects using RMN-YOLO with multi-scale edge and attention enhancements," *IEEE Access*, vol. 13, pp. 25 632–25 645, 2025.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] Hailo Technologies Ltd., "TAPPAS — Hailo's AI application development framework," 2024. [Online]. Available: https://github.com/hailo-ai/tappas

[11] Raspberry Pi Ltd., "Raspberry Pi 5 product brief," 2023. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5/

[12] S. Ramírez, "FastAPI — Modern, fast web framework for building APIs with Python," 2019. [Online]. Available: https://fastapi.tiangolo.com

[13] GStreamer Team, "GStreamer: Open source multimedia framework," 2024. [Online]. Available: https://gstreamer.freedesktop.org

[14] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.

[15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.

[16] B. Kaliski, "PKCS #5: Password-based cryptography specification version 2.0," RFC 2898, Internet Eng. Task Force, Sep. 2000.

[17] Cloudflare, Inc., "Cloudflare Tunnel documentation," 2024. [Online]. Available: https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/